

Carbon Emission as an Evaluation Metric for Contextual Bandit-Based LLM Inference Routing

Yuna Jung

Claremont McKenna College

Summer Research Program 2026

`yjung29@students.claremontmckenna.edu`

July 2026

Abstract

This study investigates whether carbon emission should be introduced as an evaluation metric for environmentally-friendly large language model (LLM) inference routing algorithms. We replicate GreenServ [1], a contextual multi-armed bandit routing framework that dynamically assigns queries to models from a pool of 16 language models to optimize the accuracy-energy tradeoff, on the Laguna high-performance computing cluster at USC/CARC. GreenServ currently evaluates routing performance using mean normalized accuracy, GPU energy consumption (Wh), and latency, but does not measure carbon emissions. We extend GreenServ by integrating CodeCarbon [2], an open-source Python library that measures actual energy consumption of GPU, CPU, and RAM and converts it to equivalent CO₂ (gCO₂) using regional grid carbon intensity. We conduct two experiments: (1) a per-model carbon benchmark measuring carbon emissions for 15 of GreenServ’s 16 language models across 10 test queries, finding that carbon scales with parameter count but is also influenced by model architecture; and (2) a live routing experiment running all 8 routing strategies across GreenServ’s full 2,500-query benchmark dataset with CodeCarbon measuring carbon per routing decision. Our goal is to produce an Accuracy vs. Carbon Emissions Pareto plot comparable to GreenServ’s existing Accuracy vs. Energy plot, and to determine whether energy consumption is a sufficient proxy for carbon emissions or whether carbon must be measured independently to accurately evaluate the environmental impact of LLM routing algorithms.

1 Introduction

The rapid growth of Large Language Models (LLM) deployment worldwide has raised concerns about the environmental cost of using AI. LLMs require significant power. A single inference call requires matrix multiplications across billions of parameters, drawing hundreds of watts from GPU hardware. At the scale of millions of daily queries across commercial LLM services, the cumulative energy and carbon footprint become substantial. A 2024 TED talk by AI researcher Sasha Luccioni [7] highlighted the environmental impact of machine learning data centers, which motivated the question of this work: can we design a routing system that directs queries to appropriately sized models, reducing environmental impacts while maintaining accuracy?

This question led to an exploration of smaller scale models called Small Language Models (SLMs). In language models, model sizes are compared by the number of parameters. Parameters are the settings that control a language model’s output, such as weights that define how much importance the model assigns to specific inputs and biases that help activate the models with flexibility when weighted inputs fall short of threshold. Small language models are generally defined in the 1–9 billion parameter range [3], while large frontier models can exceed 100 billion to 1 trillion parameters. SLMs are introduced as more efficient alternatives to large models for tasks that do not require full frontier model capability. The baseline observation here is that many real world queries are answerable by smaller, more efficient models — such as “find a recipe for tiramisu” or “help plan a week trip to Vietnam” — and routing them to a large model becomes wasteful.

During a literature review of existing routing approaches, we found that while several frameworks address computational or monetary efficiency [5, 6], few explicitly optimize for environmental metrics. GreenServ [1] is an exception: it uses a contextual multi-armed bandit (MAB) algorithm, LinUCB [4], to route queries across a pool of 16 models, optimizing a scalarized reward combining normalized accuracy and GPU energy consumption. However, GreenServ measures environmental cost only through GPU electricity usage in watt-hours (Wh), and does not account for carbon emissions.

This distinction matters because carbon emissions, water usage, and electricity are three distinct environmental impacts of data center operations [7], and optimizing for one does not necessarily optimize for the others. Moreover, the carbon footprint of a given amount of electricity depends on the combination of primary energy sources (fossil fuels, nuclear energy, renewables) of the local grid. The same hour of compute in a fossil fuel-heavy region emits far more CO₂ than in a renewables-heavy grid such as California’s.

This report attempts to present two contributions: (1) a per-model carbon measurement

study using CodeCarbon [2] on GreenServ’s 16 models, and (2) an integration of CodeCarbon into GreenServ’s live inference routing pipeline to measure carbon emissions per routing decision across all 8 routing strategies on the full 2,500-query benchmark. GreenServ optimizes routing using only two metrics, accuracy and GPU energy consumption. This study proposes carbon emissions as a third metric, to close the gap in evaluating environmental impact in LLM routing frameworks. If energy and carbon are linearly correlated across routing algorithms, energy remains a sufficient proxy. If they diverge, a new reward function incorporating carbon is needed. The central research question is: *Is GPU energy consumption a sufficient proxy for carbon emissions when evaluating the environmental impact of LLM inference routing algorithms?*

2 Background and Related Work

2.1 Environmental Costs of Language Models

The environmental impact of LLMs is measured in three primary dimensions: electricity consumption, carbon emissions, and water usage [7, 9]. These costs occur across a language model’s life cycle, from training and inference to data center storage. For example, training a series of transformer language models (OLMo, 20M to 13B parameters) released 493 metric tons of carbon emissions and consumed 2,769 million liters of water [13]. Schwartz et al. [10] introduced the concept of “Green AI,” arguing that energy efficiency should be a priority alongside accuracy. This study narrows the scope to measuring electricity usage and carbon emissions, without water consumption. While inference per query is much cheaper than training, its cumulative cost at scale is significant.

2.2 LLM and SLM Routing

Routing frameworks aim to reduce inference cost by directing queries to models of appropriate capacity. FrugalGPT [5] uses a cascade approach: queries are sent to the smallest model first, then progressively larger models until a confidence threshold is met, optimizing for API cost. RouteLLM [6] trains a classifier on human preference data to predict whether a query needs a strong or weak model. Both works use cost in dollars or token budgets as proxies for resource consumption, not direct energy or carbon measurements.

2.3 GreenServ

GreenServ [1] is the closest prior work to this study. It implements a context-aware routing framework using LinUCB [4], a contextual bandit algorithm that assumes a linear relationship between query context and model reward. The system routes queries based on three features: task type (text summarization, grade-level math, objective testable queries), semantic cluster (token embeddings showing relative closeness in meaning), and text complexity (Flesch Reading Ease score). These features are used to select among 16 models spanning 0.5B to 34B parameters. The reward function balances accuracy and energy efficiency:

$$r_t = (1 - \lambda) \cdot \text{Acc}_{\text{norm}} - \lambda \cdot E_{\text{norm}} \quad (1)$$

where $\lambda \in [0, 1]$ controls the tradeoff. GreenServ evaluates routing algorithms on mean normalized accuracy and total energy consumption (Wh), measured via the Zeus GPU monitoring library [11]. It does not measure carbon emissions.

2.4 CodeCarbon

CodeCarbon [2] is an open-source Python library that measures the carbon footprint of computational workloads. It monitors GPU, CPU, and RAM power consumption via hardware interfaces (NVIDIA NVML for GPU, Intel RAPL for CPU) and converts total energy (kWh) to CO₂ equivalent (kgCO₂eq) using the carbon intensity of the local electricity grid, sourced from regional databases or the ElectricityMap API. Two tracker modes are available: `EmissionsTracker` (online, uses real-time grid data) and `OfflineEmissionsTracker` (offline, uses a static carbon intensity database by country). Prior work [8] used CodeCarbon to benchmark the carbon footprint of HuggingFace models across tasks and hardware, demonstrating its suitability for measuring inference carbon emissions.

3 Methods

3.1 Infrastructure

All experiments were conducted on the Laguna high-performance computing cluster at the University of Southern California’s Center for Advanced Research Computing (CARC), accessed by Claremont McKenna College. The cluster uses SLURM for job scheduling. Compute nodes used in this project were equipped with dual NVIDIA L40S GPUs (46 GB VRAM each, 92 GB total) and AMD EPYC 9354 32-core processors with 128 GB RAM, which are well-suited for medium-sized language model inference. Model weights

were stored in project storage (`/project/JehoPark_1895/Yuna_SLM/`), providing 338 TB of shared network-attached storage.

Since Laguna does not support Docker for security reasons, GreenServ’s PostgreSQL database dependency was migrated to Apptainer (formerly Singularity), the HPC-standard container runtime. The migration required pulling the official PostgreSQL 15 Docker image via Apptainer, initializing the database directory in project storage, and creating a writable Unix socket directory to bypass the container’s read-only filesystem. This migration is documented in full in the lab notebook for reproducibility.

3.2 GreenServ Replication

We cloned GreenServ’s public repository (<https://github.com/TZData1/llm-inference-router>) and set up a Python 3.11 virtual environment with the provided `requirements.txt`. The PostgreSQL database was populated by loading `db/llm_db_backup.sql` into a local instance, yielding 40,000 pregenerated inference results ($2,500 \text{ queries} \times 16 \text{ models} \times \text{multiple runs}$). All four GreenServ experiments were executed via the provided Makefile targets (`make exp01-full` through `make exp04-full`), using SLURM for job management with `nohup` to keep experiments running through session disconnections.

The key challenge was that GreenServ’s experiments replay pregenerated results from the database rather than running live inference. This means neither Zeus nor CodeCarbon measures emissions during the standard experiment runs. Carbon measurements for the routing algorithm comparison therefore required a separate live inference pipeline (Section 3.5).

3.3 Experiment 1: Per-Model Carbon Benchmark

To establish an understanding of how carbon emissions vary across GreenServ’s model pool, we ran each model through 10 test queries and measured carbon emissions using CodeCarbon’s `OfflineEmissionsTracker` with `country_iso_code="USA"`. The 10 queries were general knowledge prompts (e.g., “What is the capital of France?”, “What is machine learning?”) designed to elicit short, factual responses. Of GreenServ’s 16 models, 15 were successfully measured. Yi-34B (01-ai/Yi-34B-Chat) failed due to transformer compatibility issues. Results were saved to `live_carbon_results.csv`.

3.4 CodeCarbon Practice Run (HCI-SLM)

Before integrating CodeCarbon into GreenServ’s inference pipeline, we used VedadTUG/HCI-SLM [14] as a reference implementation to learn CodeCarbon’s tracking API and validate our

measurement approach on smaller models. We ran CodeCarbon’s `OfflineEmissionsTracker` on NanoGPT and TinyLlama. This practice run informed the tracker configuration (`country_iso_code="USA"`) used in subsequent experiments.

3.5 Experiment 2: Live Routing Carbon Measurement

To measure carbon emissions *per routing algorithm* — rather than per model — we designed a live inference experiment that mirrors GreenServ’s setup but replaces the pregenerated database replay with actual model inference, with CodeCarbon measuring carbon during each inference call.

3.5.1 Routing Algorithms

We evaluated all 8 routing strategies tested in GreenServ:

- **LinUCB (C)**: GreenServ’s primary algorithm, contextual bandit [4].
- **Thompson Sampling (C)**: Bayesian posterior sampling.
- **ϵ -Greedy (C)**: Contextual epsilon-greedy.
- **ϵ -Greedy (NC)**: Non-contextual epsilon-greedy.
- **Random**: Uniformly random model selection.
- **Largest**: Always selects Yi-34B (largest model).
- **Smallest**: Always selects Qwen2.5-0.5B (smallest model).
- **Accuracy**: Always selects Gemma-3-27B (highest accuracy model).

3.5.2 Feature Extraction

We used GreenServ’s full feature extraction pipeline: task type (derived from the source dataset label — MMLU, GSM8K, HellaSwag, WinoGrande, CNN/DailyMail — one-hot encoded), semantic cluster (sentence embeddings, one-hot encoded), and text complexity (Flesch Reading Ease score, one-hot encoded).

3.5.3 Reward Computation

To evaluate routing decisions without re-running model evaluation, we used GreenServ’s pregenerated accuracy and energy values from the database as the reward signal, consistent with GreenServ’s offline evaluation approach [1].

3.5.4 Carbon Measurement

For each algorithm, we wrapped the full 2,500-query inference run in a single CodeCarbon `OfflineEmissionsTracker` instance. This captures the total GPU, CPU, and RAM energy consumed by that algorithm’s routing decisions, which implicitly reflects model selection frequency. When an algorithm like LinUCB primarily routes to small models, it consumes less GPU power; when the Largest baseline always calls Yi-34B, GPU power draw is consistently high. CodeCarbon [2] captures these differences at the hardware level.

3.5.5 Errors Encountered

A key implementation challenge was GPU out-of-memory errors between algorithms. Models loaded during one algorithm’s run remained in GPU memory and caused the next algorithm’s model loads to fail. We resolved this by explicitly calling `del model`, `del tokenizer`, `torch.cuda.empty_cache()`, and `gc.collect()` after each algorithm’s run before starting the next. Due to time constraints, the outcome of this fix was not fully verified; completing and validating the 8-algorithm comparison remains future work.

4 Results

4.1 GreenServ Replication

As seen in Figure 1, the red dotted Pareto front line connects the most efficient algorithm run combinations — those achieving the best accuracy for a given energy level. LinUCB and Thompson Sampling lie on or near this front, meaning they achieve near-best accuracy at significantly lower energy than always using the largest or most accurate model. This is GreenServ’s central claim: intelligent routing reduces environmental cost without sacrificing accuracy. However, this plot only captures one dimension of environmental impact: electricity consumption measured in watt-hours by Zeus [11].

The goal of this study is to produce an equivalent plot with carbon emissions (gCO_2) on the x-axis instead of watt-hours. If the two plots look identical, meaning the same algorithms that minimize energy also minimize carbon, then watt-hours is a sufficient metric for evaluating environmental impact in routing systems. If the plots differ, it would suggest that some algorithms consume relatively low GPU energy but disproportionately high carbon, or vice versa. This could happen because CodeCarbon measures CPU and RAM energy in addition to GPU energy, and because the local grid’s carbon intensity converts the same watt-hour differently depending on whether it came from renewables or fossil fuels. In that

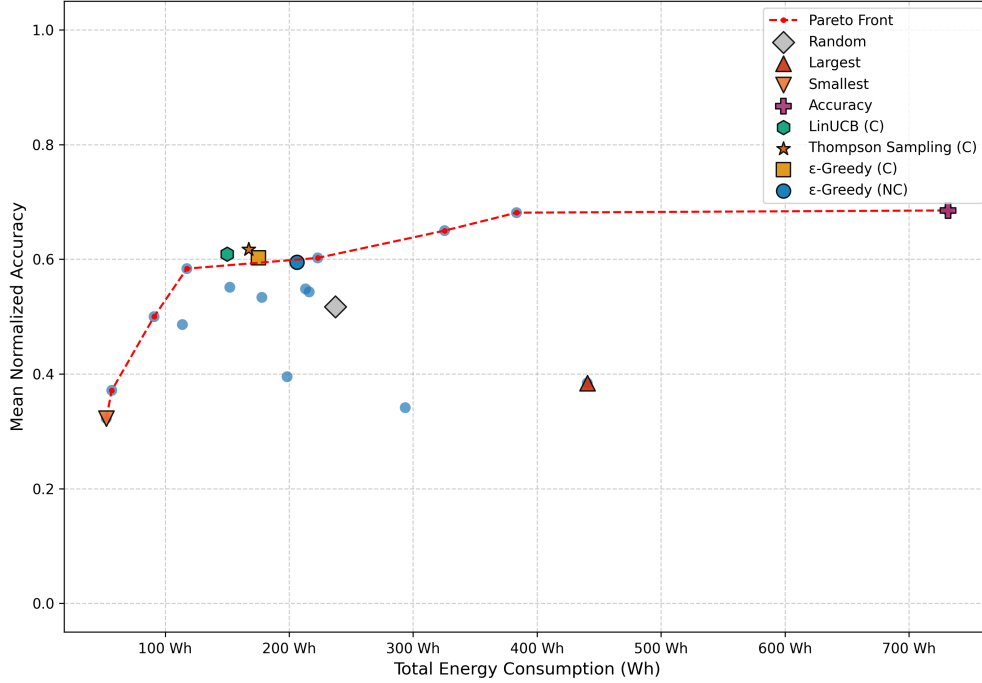


Figure 1: Replication of Pregenerated Experiments with GreenServ

case, GreenServ’s existing reward function would need to be updated to incorporate carbon directly, rather than using energy as a proxy.

To study this, we conducted two experiments.

4.2 Experiment 1: Per-Model Carbon Benchmark

Figure 2 and Table 1 show carbon emissions per model for 10 queries each, measured on the Laguna L40S GPU using `OfflineEmissionsTracker` [2].

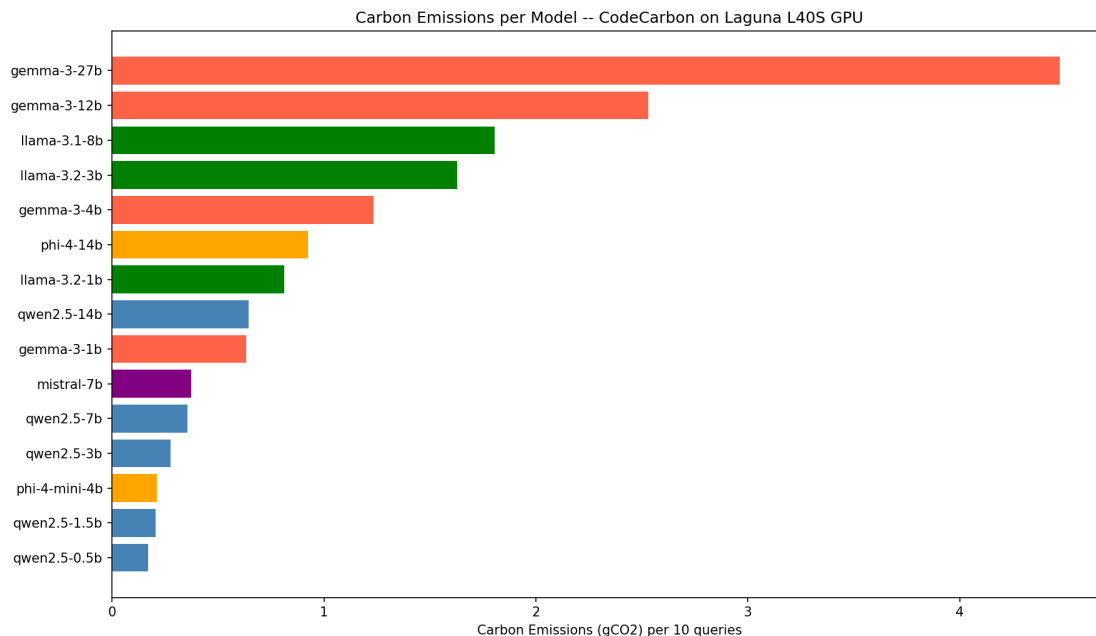


Figure 2: Carbon emissions per model (10 queries, Laguna L40S GPU)

Table 1: Carbon emissions per model (10 queries, Laguna L40S GPU)

Model	Parameters	Carbon (gCO ₂)
Qwen2.5-0.5B	0.5B	0.171
Qwen2.5-1.5B	1.5B	0.207
Phi-4-mini-4B	3.8B	0.213
Qwen2.5-3B	3B	0.221
Qwen2.5-7B	7B	0.355
Mistral-7B	7B	0.373
Gemma-3-1B	1B	0.634
Qwen2.5-14B	14B	0.646
Phi-4-14B	14B	0.925
Gemma-3-4B	4B	1.234
Llama-3.1-8B	8B	1.807
Llama-3.2-3B	3B	≈1.6
Llama-3.2-1B	1B	≈0.8
Gemma-3-12B	12B	2.529
Gemma-3-27B	27B	4.470
Yi-34B	34B	[failed]

Looking at the Gemma-3 family, the highest parameter count (27B) has the highest carbon emission, and carbon emissions generally increase with parameter count. However, model architecture also affects carbon efficiency: Phi-4-mini (3.8B parameters) emits less carbon per query than Qwen2.5-1.5B (1.5B parameters), suggesting that architectural efficiency differences can be more significant than raw parameter counts in some cases. Similarly, Gemma-3 models emit substantially more carbon than Qwen2.5 models of similar size.

4.3 Experiment 2: Live Routing Carbon Measurement

The live routing experiment was submitted to Laguna’s GPU partition multiple times across several sessions but did not complete within the timeframe of this research project. The experiment encountered model loading overhead: the time required to load model weights into GPU memory when a routing algorithm switches from one model to another. With 2,500 queries and 16 models, a routing algorithm that switches frequently could trigger hundreds of model loads. Each load takes 30–120 seconds, making the experiment potentially require hours of loading time before inference begins. This makes the experiment infeasible within a single SLURM job.

In GreenServ’s original design [1], all model inference results are pregenerated and stored in a PostgreSQL database. During experiments, the router replays these results without loading model weights, making GreenServ’s experiments fast and GPU-memory-efficient. Our live routing experiment departed from this design, which is necessary to measure real CodeCarbon emissions but introduces the overhead described above.

5 Discussion and Future Work

The central research question — whether GPU energy consumption is a sufficient proxy for carbon emissions when evaluating the environmental impact of LLM inference routing algorithms — remains unanswered. The live routing experiment (Section 3.5) did not complete within the project timeframe. To overcome model loading overhead, two approaches are viable:

1. **Preload all models simultaneously:** Load all 15 models into GPU memory at once, eliminating reload overhead. Preloading all models in `bf16` precision requires approximately 150–180 GB of VRAM. Laguna’s dual L40S nodes provide 92 GB total, which is insufficient. This would require either a multi-GPU node with higher total VRAM, or hardware such as NVIDIA H100 80GB GPUs.

⁰Llama-3.2 models did not complete the full benchmark run within the project timeframe.

2. **Two-phase carbon attribution:** Run each model separately on all 2,500 queries with CodeCarbon measuring carbon per model per query (Phase 1, partially completed — 9 of 15 models), then use GreenServ’s pregenerated routing decisions to reconstruct which model each algorithm would have chosen per query and sum the corresponding carbon measurements (Phase 2). This avoids model switching entirely and is the most feasible path forward given available hardware.

Future experiments could also measure the water usage of language models alongside energy consumption and carbon footprint, since data center cooling accounts for substantial water use that is not captured by carbon metrics alone [7].

Finally, because this experiment was run in Southern California, results are shaped by that region’s electricity grid carbon intensity. California draws a comparatively high share of renewable energy relative to other states. Repeating this experiment with different regional carbon intensity codes would allow for meaningful comparison. Virginia, for example, is widely reported to be the world’s largest hub for data centers, with a grid mix relying more heavily on natural gas and nuclear power. Investigating these regional differences could inform public policy decisions regarding data center siting and regulation, and how different grid mixes affect the environmental impact of AI inference.

6 Conclusion

This study extends GreenServ [1], a contextual bandit-based LLM inference routing framework, by integrating CodeCarbon [2] to measure carbon emissions alongside the existing GPU energy metric. We demonstrate that carbon emissions per model scale with parameter count but are also meaningfully influenced by model architecture — a finding with direct implications for environmentally-conscious model pool design in routing frameworks. The live routing experiment was designed to produce an Accuracy vs. Carbon Emissions Pareto plot comparable to GreenServ’s existing Accuracy vs. Energy plot, but could not be completed within the project timeframe due to GPU memory and model loading constraints. This limitation itself is a finding: integrating real-time carbon measurement into a live routing system requires architectural choices that energy-only measurement with Zeus does not. Future work should complete the two-phase carbon attribution approach and investigate whether energy consumption and carbon are linearly correlated across routing algorithms, or whether carbon must be treated as an independent evaluation metric.

Acknowledgments

This research was conducted as part of the Claremont McKenna College Summer Research Program 2026 at the Murty Sunak Quantitative Computing Lab, under the supervision of Professor Jeho Park. Computing resources were provided by the USC Center for Advanced Research Computing (CARC) Laguna cluster. There was generative AI assistance in this work for coding, logging, and creating the report, under supervision and approval of the supervising professor.

References

- [1] Ziller, T., et al. *GreenServ: Energy-Efficient LLM Inference Routing via Contextual Bandits*. 2026.
- [2] Courty, B., et al. *CodeCarbon: Estimate and Track Carbon Emissions from Machine Learning Computing*. <https://github.com/mlco2/codecarbon>, 2023.
- [3] Gupta, A., et al. *A Survey of Small Language Models in the 1–9 Billion Parameter Range*. Amazon Machine Learning, 2026.
- [4] Li, L., et al. *A Contextual-Bandit Approach to Personalized News Article Recommendation*. In *WWW*, 2010.
- [5] Chen, L., et al. *FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance*. arXiv:2305.05176, 2023.
- [6] Ong, I., et al. *RouteLLM: Learning to Route LLMs with Preference Data*. arXiv:2406.18665, 2024.
- [7] Luccioni, A.S. *The Environmental Impact of AI*. TED Talk, 2023.
- [8] Luccioni, A.S., Viguiet, S., Ligozat, A.L. *Estimating the Carbon Footprint of BLOOM, a 176B Parameter Language Model*. *JMLR*, 2023.
- [9] Strubell, E., Ganesh, A., McCallum, A. *Energy and Policy Considerations for Deep Learning in NLP*. In *ACL*, 2019.
- [10] Schwartz, R., et al. *Green AI. Communications of the ACM* 63, 12 (2020), 54–63.
- [11] You, J., et al. *Zeus: Understanding and Optimizing GPU Energy Consumption of DNN Training*. In *NSDI*, 2023.

- [12] Faiz, A., et al. *LLMCarbon: Modeling the End-to-End Carbon Footprint of Large Language Models*. arXiv:2309.14393, 2023.
- [13] Morrison, A., et al. *Environmental Impacts of OLMo: A Case Study in Open Language Model Training*. 2025.
- [14] VedadTUG. *HCI-SLM: Comparing Energy Consumption of Selected Small Language Models*. <https://github.com/VedadTUG/HCI-SLM>, 2024.